

ACTIVIDAD 10: TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE

PROGRAMACION WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO

Contenido

1. NIVEL.....	2
2. TIPO DE EXPLOTACIÓN.....	2
3. OBJETIVO DEL LABORATORIO	2
4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD	2
5. PROCEDIMIENTO PASO A PASO	2
5.1 Acceso al laboratorio	3
5.2 Identificación de la solicitud	3
5.3 Envío de la solicitud a Repeater.....	3
5.4 Modificación del parámetro vulnerable	4
5.5 Envío de la solicitud modificada	4
5.6 Análisis de la respuesta.....	4
5.7 Confirmación de la resolución	4
6. EVIDENCIAS.....	5
7. EXPLICACIÓN DEL PAYLOAD	8
8. MITIGACIÓN RECOMENDADA.....	9
CONCLUSIÓN	9

1. NIVEL

Practitioner

2. TIPO DE EXPLOTACIÓN

Path Traversal mediante evasión de un filtro utilizando doble URL encoding.

3. OBJETIVO DEL LABORATORIO

El objetivo del laboratorio es explotar una vulnerabilidad de **Path Traversal** presente en la funcionalidad encargada de mostrar las imágenes de los productos.

La aplicación bloquea las entradas que contienen secuencias de recorrido de directorios, como `../`, y posteriormente realiza una operación de **URL decode** sobre el valor proporcionado antes de utilizarlo para acceder al archivo.

Debido al orden en que se realizan estas operaciones, es posible utilizar una representación codificada de la secuencia de recorrido para evadir el filtro inicial y conseguir que, después del proceso de decodificación, la entrada se convierta nuevamente en una ruta de recorrido.

El objetivo final es acceder al archivo:

`/etc/passwd`

y demostrar que es posible obtener información ubicada fuera del directorio destinado al almacenamiento de las imágenes.

4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

La vulnerabilidad se produce porque la aplicación utiliza información proporcionada por el usuario para construir la ruta de un archivo y aplica un filtro sobre la entrada antes de realizar una operación posterior de **URL decoding**.

En condiciones normales, una solicitud como:

`../../etc/passwd`

contiene directamente las secuencias `../`, por lo que la aplicación puede detectarlas y bloquearlas.

Sin embargo, el laboratorio realiza posteriormente una decodificación URL. Esto permite representar determinados caracteres mediante una codificación adicional y evitar que la secuencia de recorrido sea reconocida durante la primera validación.

El payload utilizado fue:

`..%252f..%252f..%252fetc/passwd`

El funcionamiento puede representarse de la siguiente manera:

..%252f..%252f..%252fetc/passwd

↓

Primera decodificación URL

..%2f..%2f..%2fetc/passwd

↓

Segunda interpretación/decodificación

../../etc/passwd

De esta manera, los caracteres / que forman parte de la ruta de recorrido no aparecen directamente en la entrada inicial como una secuencia ../.

El problema principal no consiste únicamente en la existencia de un filtro insuficiente, sino en la combinación del filtrado con una operación posterior de **URL decoding**. Si la validación se realiza antes de que la entrada alcance su forma final, un atacante puede proporcionar una representación equivalente que posteriormente se transforme en una ruta peligrosa.

Este comportamiento demuestra por qué la validación de rutas debe realizarse sobre la representación final normalizada de la entrada y no únicamente sobre una representación previa.

5. PROCEDIMIENTO PASO A PASO

5.1 Acceso al laboratorio

Se accedió al laboratorio de **File Path Traversal, traversal sequences stripped with superfluous URL-decode** de PortSwigger Web Security Academy.

La funcionalidad vulnerable se encuentra en la solicitud utilizada para obtener las imágenes de los productos.

5.2 Identificación de la solicitud

Mediante Burp Suite se interceptó la solicitud correspondiente a una imagen de producto.

La solicitud original fue:

GET /image?filename=5.jpg HTTP/2

El parámetro vulnerable identificado fue:

filename

Este parámetro controla el nombre del archivo que la aplicación intenta recuperar.

5.3 Envío de la solicitud a Repeater

La solicitud fue enviada a **Burp Suite Repeater** para poder modificar el parámetro filename y analizar la respuesta del servidor.

5.4 Modificación del parámetro vulnerable

Se substituyó el valor original del parámetro:

filename=5.jpg

por el siguiente payload:

filename=..%252f..%252f..%252fetc/passwd

La solicitud modificada quedó de la siguiente manera:

GET /image?filename=..%252f..%252f..%252fetc/passwd HTTP/2

5.5 Envío de la solicitud modificada

Al enviar la solicitud modificada, el servidor respondió con:

HTTP/2 200 OK

La respuesta contenía el contenido del archivo /etc/passwd.

Entre los datos obtenidos se encontraba la entrada:

root:x:0:0:root:/root:/bin/bash

Esto confirmó que el proceso de decodificación permitió que la entrada adquiriera una representación equivalente a una ruta de Path Traversal hacia /etc/passwd, haciendo posible recuperar el contenido del archivo.

5.6 Análisis de la respuesta

El cuerpo de la respuesta contenía el contenido del archivo /etc/passwd, comenzando con la entrada correspondiente al usuario root.

root:x:0:0:root:/root:/bin/bash

También se observaron otras cuentas del sistema, confirmando que la aplicación había permitido acceder al archivo solicitado.

5.7 Confirmación de la resolución

Después de comprobar que el contenido de /etc/passwd había sido recuperado correctamente, se regresó a la interfaz del laboratorio.

El laboratorio mostró el mensaje:

Lab Solved

confirmando que la explotación había sido realizada correctamente.

6. EVIDENCIAS

Figura 1. Solicitud HTTP original.

Descripción

Se muestra la solicitud HTTP original interceptada mediante Burp Suite. La petición corresponde a la carga de una imagen del producto y utiliza el parámetro filename con el valor 5.jpg. Esta solicitud representa el comportamiento normal de la aplicación antes de realizar cualquier modificación.

Evidencia:

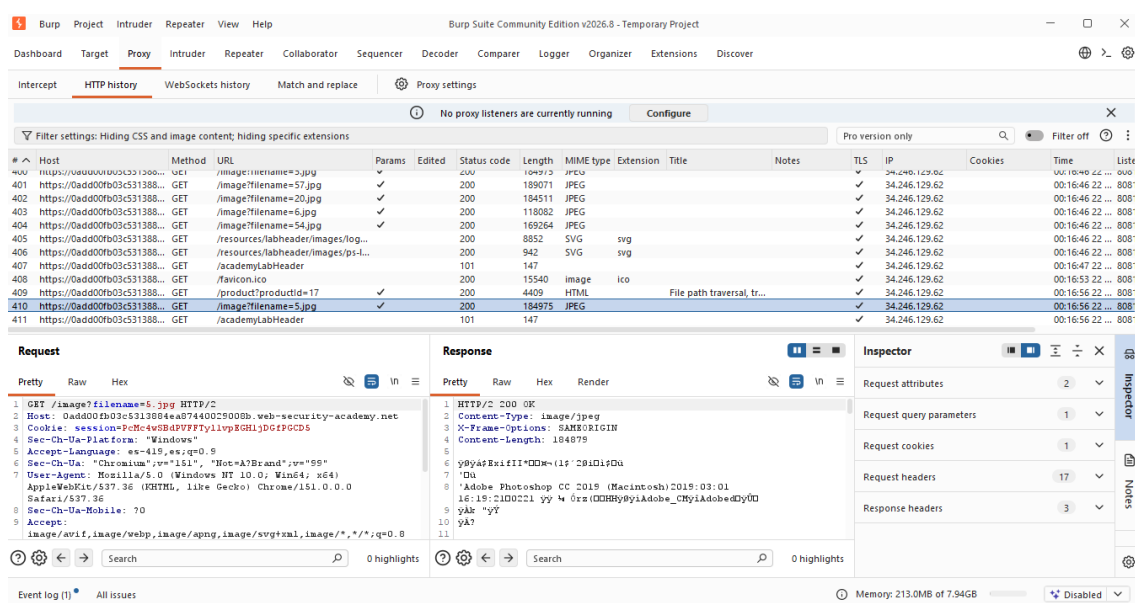


Figura 2. Payload utilizado

Descripción:

Se sustituyó el valor original del parámetro filename por **..%252f..%252f..%252fetc/passwd**. Este payload utiliza doble URL encoding para representar de forma codificada los caracteres / que forman las secuencias de recorrido. De esta manera, la entrada puede superar el filtro inicial y posteriormente ser transformada mediante el proceso de URL decoding hasta adquirir la forma de una ruta de Path Traversal.

Evidencia:

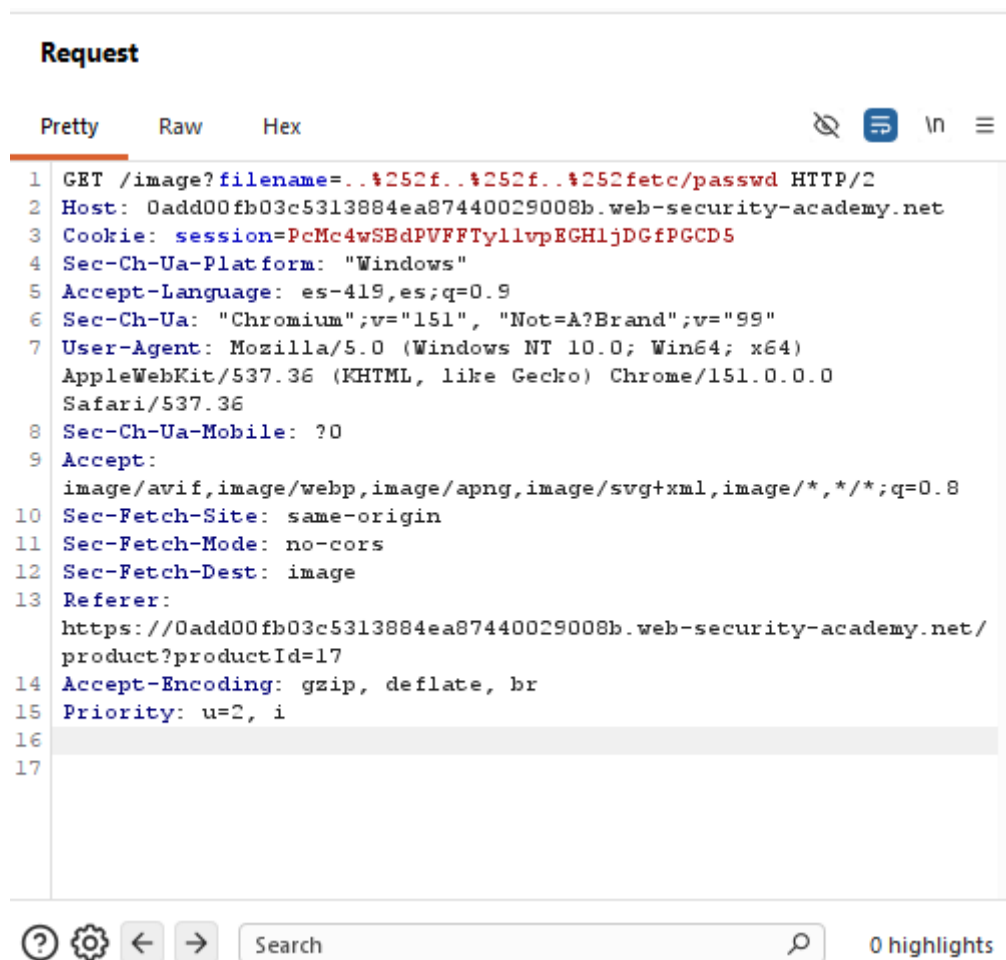
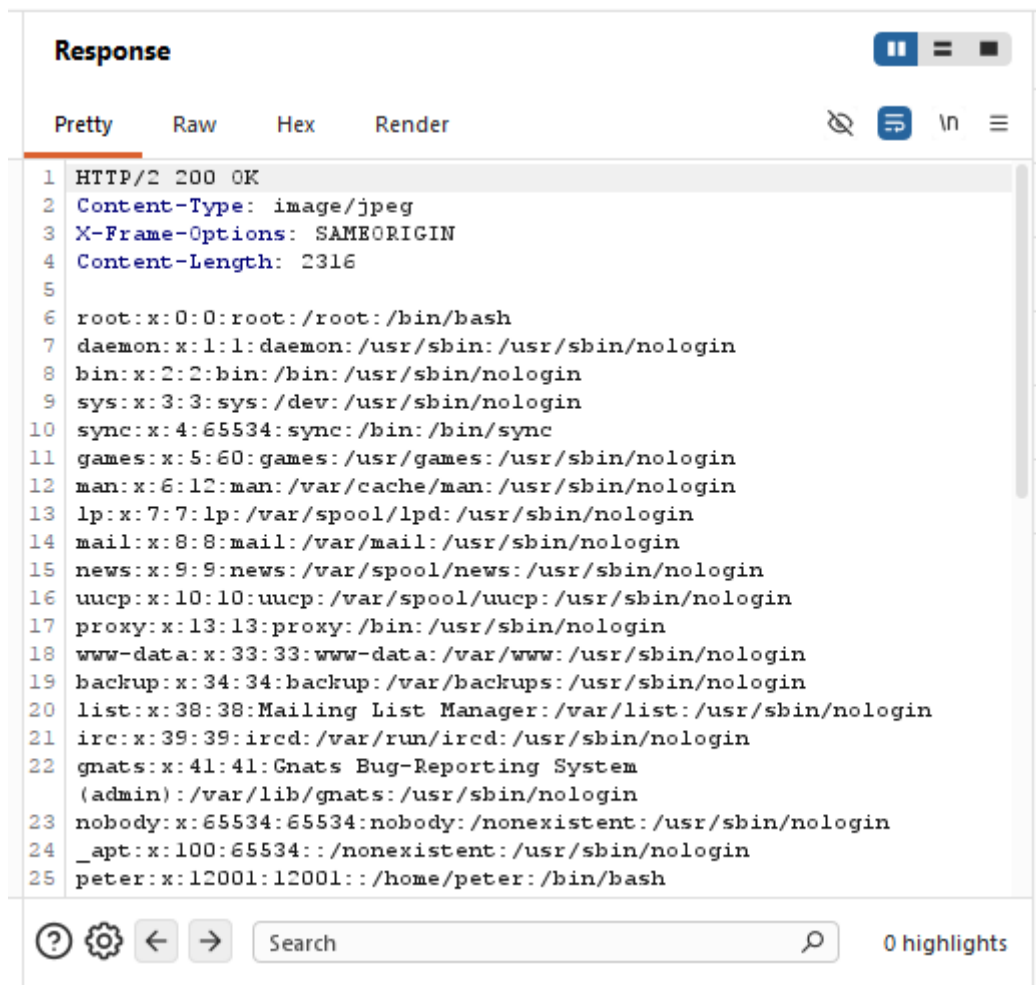


Figura 3. Respuesta vulnerable

Descripción:

Se muestra la respuesta obtenida después de enviar la solicitud modificada. El servidor respondió con **HTTP/2 200 OK** y, a pesar de que la petición correspondía a la carga de una imagen, el cuerpo de la respuesta contiene el contenido del archivo `/etc/passwd`. La presencia de entradas como `root:x:0:0:root:/root:/bin/bash` confirma que el servidor procesó la ruta manipulada y permitió acceder al archivo solicitado.

Evidencia:



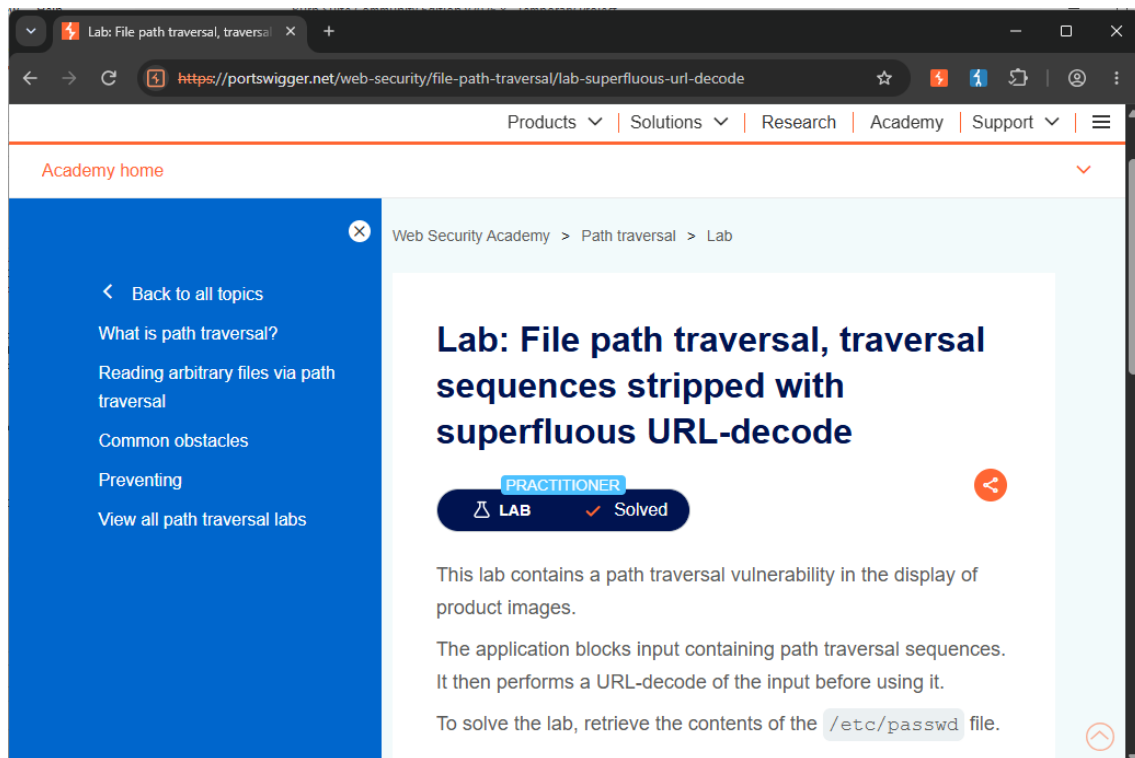
```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ired:/var/run/ired:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
  (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534:./nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:./home/peter:/bin/bash
```

Figura 4. Confirmación de Lab Solved

Descripción:

Se muestra la confirmación proporcionada por PortSwigger Web Security Academy después de ejecutar correctamente la explotación. El estado **Lab Solved** confirma que el acceso al archivo `/etc/passwd` fue reconocido por la plataforma y que el objetivo establecido para el laboratorio fue alcanzado.

Evidencia:



7. EXPLICACIÓN DEL PAYLOAD

El payload utilizado fue:

..%252f..%252f..%252fetc/passwd

Su funcionamiento depende del proceso de **URL decoding** realizado por la aplicación.

La secuencia %25 representa el carácter %. Por lo tanto, una secuencia como:

%252f

puede convertirse primero en:

%2f

y posteriormente interpretarse como:

/

El proceso completo puede representarse conceptualmente como:

..%252f..%252f..%252fetc/passwd

↓

..%2f..%2f..%2fetc/passwd

↓

../../etc/passwd

La ventaja del payload es que la solicitud inicial representa los caracteres / mediante una codificación adicional, por lo que la secuencia ../ no aparece directamente en su forma original durante la validación inicial. Posteriormente, el proceso de URL decoding transforma progresivamente la entrada hasta recuperar los caracteres utilizados en la ruta de recorrido.

Después de la decodificación, los caracteres recuperan su significado original y la ruta resultante permite realizar el recorrido de directorios.

Por lo tanto, el ataque aprovecha una diferencia entre la representación utilizada durante la validación y la representación utilizada posteriormente para acceder al archivo.

8. MITIGACIÓN RECOMENDADA

Para evitar vulnerabilidades de Path Traversal de este tipo, se recomienda:

- Evitar utilizar directamente nombres de archivos proporcionados por el usuario para construir rutas del sistema de archivos.
- Utilizar una **lista de permitidos (allowlist)** de archivos o identificadores válidos cuando sea posible.
- Realizar la decodificación y normalización de la entrada antes de efectuar cualquier validación de seguridad.
- Validar la ruta final normalizada y comprobar que permanezca dentro del directorio autorizado.
- No depender únicamente de filtros de cadenas como ../, debido a que una misma ruta puede representarse de diferentes formas.
- Utilizar APIs seguras para la manipulación de archivos y rutas.
- Aplicar el principio de **mínimo privilegio** a la cuenta utilizada por la aplicación.
- Restringir los permisos de lectura del proceso para evitar que una vulnerabilidad permita acceder a archivos sensibles del sistema.

Una estrategia de mitigación adecuada debe validar el **resultado final normalizado de la ruta**, en lugar de intentar identificar manualmente todas las posibles representaciones de una secuencia de recorrido.

CONCLUSIÓN

El laboratorio permitió demostrar una vulnerabilidad de **Path Traversal** causada por una validación insuficiente combinada con un proceso posterior de URL decoding.

Aunque la aplicación bloqueaba directamente las secuencias de recorrido, fue posible evadir esta restricción mediante el uso de **doble URL encoding**. El payload `..%252f..%252f..%252fetc/passwd` fue decodificado progresivamente hasta producir una representación equivalente a `../..../etc/passwd`, permitiendo acceder al archivo `/etc/passwd`.

La respuesta HTTP/2 200 OK y la recuperación del contenido de `/etc/passwd` confirmaron la explotación.

El ejercicio demuestra la importancia de realizar las validaciones de seguridad sobre la representación final normalizada de una entrada y de no depender exclusivamente de filtros basados en cadenas específicas.